

Nifty100 Financial Intelligence Platform

Analyst Guide

Version: 1.0

Author: Shashank Charpe

Project: Nifty100 Financial Intelligence Platform

Technology Stack: Python | Streamlit | FastAPI | SQLite | Pandas | Plotly

Table of Contents

1. Introduction
2. Project Objectives
3. System Architecture
4. Technology Stack
5. Installation and Setup
6. Project Structure
7. Database Design
8. Dashboard Modules
9. REST API
10. Financial Analytics
11. Report Generation
12. Testing
13. Performance Optimization
14. Troubleshooting
15. Future Enhancements
16. Conclusion

1. Introduction

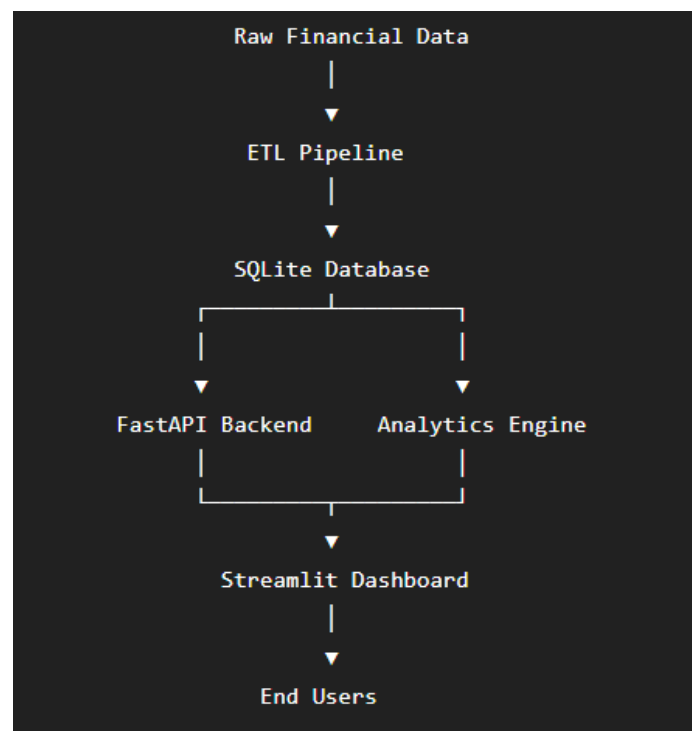
The Nifty100 Financial Intelligence Platform is a financial analytics application developed using Python. The platform provides investors and analysts with interactive dashboards, company financial analysis, stock screening, peer comparison, valuation metrics and downloadable reports. The application integrates financial datasets into a centralized SQLite database and presents insights through Streamlit dashboards and FastAPI endpoints.

2. Project Objectives

The primary objectives of this project are:

- Build a centralized financial analytics platform.
- Analyze financial statements of Nifty100 companies.
- Provide interactive dashboards.
- Compare companies with peers.
- Calculate financial KPIs.
- Generate downloadable reports.
- Provide REST APIs for integration.

3. System Architecture



The architecture separates data ingestion, analytics, APIs, and visualization into independent modules.

4. Technology Stack

Programming Language

- Python 3.10

Libraries

- Pandas
- NumPy
- Plotly
- Streamlit
- FastAPI
- ReportLab
- OpenPyXL

Database

- SQLite

Testing

- PyTest

Code Quality

- Black
- Ruff

5. Installation and Setup

Clone the repository:

```
git clone <repository-url>
```

Move to the project directory:

```
cd nifty100-platform
```

Install dependencies:

```
pip install -r requirements.txt
```

Run the dashboard:

```
streamlit run src/dashboard/app.py
```

Run the API:

```
uvicorn src.api.main:app --reload
```

6. Project Structure

nifty100-platform/

├── data/

├── docs/

├── output/

├── screenshots/

├── src/

| ├── analytics/

| ├── api/

| ├── dashboard/

| ├── etl/

| ├── reports/

| └── scripts/

├── tests/

├── README.md

└── requirements.txt

Each module is organized to improve maintainability and scalability.

7. Database Design

The application stores financial information inside a SQLite database.

Major tables include:

- Companies
- Profit & Loss
- Balance Sheet
- Cash Flow
- Financial Ratios
- Market Capitalization
- Stock Prices
- Sector Information

The database is indexed to improve dashboard performance.

8. Dashboard Modules

Home Dashboard : Provides an overview of the financial platform.

Company Profile : Displays company information and financial metrics.

Stock Screener : Filters companies using predefined financial criteria.

Peer Comparison : Compares selected companies using financial indicators.

Trend Analysis : Displays historical financial trends.

Sector Analysis : Compares companies within the same sector.

Capital Allocation : Visualizes how companies allocate capital.

Annual Reports : Provides downloadable company reports.

Valuation Analytics : Calculates valuation metrics and performance indicators.

9. REST API

The application exposes FastAPI endpoints.

Examples include:

GET /health

GET /api/v1/companies

GET /api/v1/companies/{ticker}

GET /api/v1/screener

GET /api/v1/sectors/{sector}/companies

These APIs allow external systems to consume financial data.

10. Financial Analytics

The analytics module calculates important financial KPIs.

Examples include:

Revenue CAGR

PAT CAGR

EPS CAGR

Debt to Equity

Return on Equity

Operating Margin

Net Profit Margin

Free Cash Flow

CFO Quality Score

CAPEX Intensity

These metrics help evaluate financial health and business performance.

11. Report Generation

The platform generates:

- Company PDF Tearsheet
- Valuation Summary
- Performance Reports
- Financial Reports
- CSV Exports
- Excel Reports

Reports can be shared with analysts and management.

12. Testing

The project uses PyTest for automated testing.

Execute tests:

```
python -m pytest tests -q
```

Current status:

79 Passed

The project also includes API tests, analytics tests, ETL tests, and dashboard-related validations.

13. Performance Optimization

The project includes several optimizations:

- SQLite indexes
- Cached database queries
- Optimized data loading
- Efficient dashboard rendering
- Performance benchmarking

These improvements reduce response time and improve user experience.

14. Troubleshooting Guide

This section provides solutions to common issues that users may encounter while running the Nifty100 Financial Intelligence Platform.

Database Connection Issues

Problem: The dashboard displays an error such as *"Unable to connect to database"*.

Possible Causes:

- The nifty100.db file is missing.
- Incorrect database path in the application.
- SQLite database is locked by another process.

Solution:

- Verify that output/nifty100.db exists.
- Ensure the correct database path is configured.
- Close any application using the database and restart the dashboard.

API Server Not Starting

Problem: FastAPI server fails to start.

Possible Causes:

- Required Python packages are not installed.
- Port 8000 is already occupied.
- Syntax or import errors in the API code.

Solution:

- Install project dependencies using:

```
pip install -r requirements.txt
```

- Check if port 8000 is already in use.
- Review the terminal output for detailed error messages.

Dashboard Loading Slowly

Problem: Dashboard takes a long time to load.

Possible Causes:

- Large SQL queries.
- Missing database indexes.
- System resource limitations.

Solution:

- Run the database indexing script.
- Close unnecessary applications.
- Ensure the latest optimized database is being used.

Empty Charts or Missing Data

Problem: Charts appear blank or no records are displayed.

Possible Causes:

- Missing data in the database.
- Incorrect company selection.
- Data import process was not completed successfully.

Solution:

- Verify that ETL completed without errors.
- Check the selected company.
- Confirm that the required tables contain data.

PDF Tearsheet Generation Failed

Problem: PDF report is not generated.

Possible Causes:

- Missing ReportLab library.
- Invalid company data.
- Missing output directory.

Solution:

- Install ReportLab.
- Verify company records exist.
- Ensure the reports/tearsheets directory is available.

Common Commands

Task	Command
Start API	<code>uvicorn src.api.main:app --reload</code>
Start Dashboard	<code>streamlit run src/dashboard/Home.py</code>
Run Tests	<code>pytest</code>
Generate Reports	<code>python src/reports/tearsheet.py</code>

15. Future Enhancements

Although the platform successfully fulfills the current project requirements, several improvements can further enhance its capabilities and scalability.

Live Market Data Integration

Currently, the platform works with historical financial datasets. In future versions, APIs from NSE or other financial data providers can be integrated to display real-time stock prices and market updates.

PostgreSQL Migration

SQLite is suitable for development and lightweight applications. For larger datasets and multiple concurrent users, migrating to PostgreSQL or MySQL would improve scalability and performance.

User Authentication

Future versions can include secure user authentication with role-based access.

Possible roles include:

- Administrator

- Financial Analyst
- Read-Only User

This will improve data security and access control.

Advanced Financial Analytics

Additional financial indicators can be incorporated, including:

- Sharpe Ratio
- Altman Z-Score
- Piotroski F-Score
- EV/EBITDA
- PEG Ratio
- Monte Carlo Risk Simulation

These metrics would provide deeper investment insights.

Interactive Visualizations

Future releases may include:

- Drill-down dashboards
- Interactive trend analysis
- Dynamic peer comparison
- Forecasting visualizations
- AI-generated financial summaries

Automated ETL Pipeline

The current ETL process is manually executed. It can be automated using scheduled jobs or workflow orchestration tools such as Apache Airflow to keep the database updated without manual intervention.

Cloud Deployment

The application can be deployed on cloud platforms such as:

- Microsoft Azure
- AWS
- Google Cloud Platform

Cloud deployment would provide better availability, scalability, and easier maintenance.

AI-Based Recommendation System

A future enhancement could include an AI-powered recommendation engine that analyzes financial ratios, growth trends, and peer comparisons to generate investment insights such as Buy, Hold, or Watchlist recommendations. These suggestions would serve as analytical support rather than financial advice.

16. Conclusion

The Nifty100 Financial Intelligence Platform provides a complete environment for financial analysis and visualization. By combining ETL pipelines, a centralized SQLite database, FastAPI services, Streamlit dashboards, and automated testing, the project delivers a scalable solution for analyzing Nifty100 companies. The modular architecture also allows future enhancements and easy maintenance.

Author : SHASHANK CHARPE

MCA Graduate

Java Backend Developer | Python | Data Analytics

GitHub: github.com/shashank12125

LinkedIn: linkedin.com/in/shashankcharpe